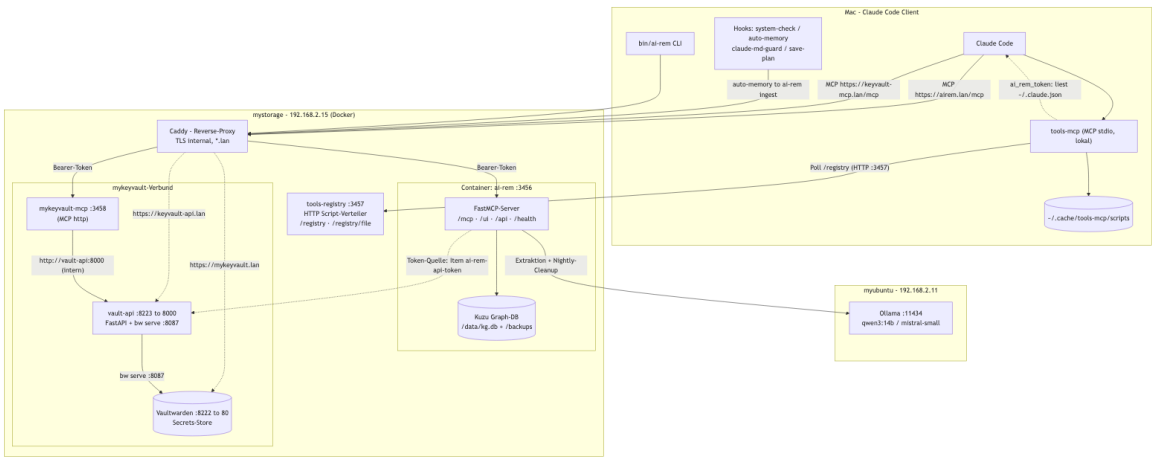


Architektur: ai-rem · mykeyvault · tools-mcp

Drei zusammengehörige MCP-Systeme für den Claude-Code-Betrieb im LAN:

- **ai-rem** — persistentes Langzeit-Gedächtnis (Knowledge Graph) als FastMCP-Server mit eingebetteter Kuzu-DB.
- **mykeyvault** — Secrets-Verbund: Vaultwarden als Store, `vault-api` als Token-authentifiziertes REST-Gateway, `mykeyvault-mcp` als MCP-Frontend.
- **tools-mcp** — lokaler MCP-Server, der Scripts als Tools registriert und sie live von einem zentralen HTTP- `tools-registry` bezieht.

Sie spielen so zusammen: Alle HTTP-MCP-Kanäle laufen über **Caddy** (TLS internal, `*.lan`) auf **mystorage**; authentifiziert wird mit **einem gemeinsamen Bearer-Token** (`ai-rem-api-token`), das ursprünglich aus Vaultwarden stammt und über `vault-api` verteilt wird. **Ollama** läuft separat auf **myubuntu** (GPU) und liefert ai-rem die Transcript-Extraktion und die nächtlichen Cleanup-Urteile. **tools-mcp** läuft als stdio-Prozess lokal auf dem Mac und synchronisiert seine Scripts per HTTP vom `tools-registry`.



Komponenten

Komponente	Host	Port	Protokoll / Zugang	Zweck
ai-rem	mystorage	3456	HTTP-MCP via <code>https://ai-rem.lan/mcp</code> (Caddy), Bearer	Knowledge-Graph-Gedächtnis (FastMCP + Kuzu), Web-UI, Backup/Cleanup
Kuzu Graph-DB	mystorage	—	eingebettet in ai-rem	Entities/Relations (<code>/data/kg.db</code>), Backups (<code>/backups</code>)
		3458		

Komponente	Host	Port	Protokoll / Zugang	Zweck
mykeyvault-mcp	mystorage		HTTP-MCP via <code>https://keyvault-mcp.lan/mcp</code> (Caddy), Bearer	MCP-Frontend für die Vault-Tools (kein Secret-Leak in den Kontext)
vault-api	mystorage	8223→8000	REST via <code>https://keyvault-api.lan</code> (Caddy), Bearer	Token-Gateway um die Bitwarden-CLI; hält <code>bw serve (:8087)</code> entsperrt
Vaultwarden	mystorage	8222→80	<code>https://mykeyvault.lan</code> (Caddy)	Eigentlicher Secrets-Store
tools-registry	mystorage	3457	reines HTTP (LAN-only, keine Auth)	Verteilt die Scripts (<code>/registry</code> , <code>/registry/file</code>)
tools-mcp	Mac (lokal)	—	MCP stdio (Node-Prozess)	Registriert Scripts als Tools; pollt den Registry alle 5 s
Ollama	myubuntu	11434	HTTP	Transcript-Extraktion + Nightly-Cleanup-Urteile für ai-rem
Caddy	mystorage	—	Reverse-Proxy, <code>tls internal</code>	Terminiert TLS für alle <code>*.lan</code> -Endpunkte

Auth: ai-rem und mykeyvault-mcp teilen sich denselben Bearer-Token (`ai-rem-api-token`, als `MCP_AUTH_TOKEN`); `vault-api` verwendet ihn als `VAULT_API_TOKEN`. Der Token stammt aus Vaultwarden und wird über `vault-api` an die Clients verteilt; ai-rem frischt ihn pro Session im Hintergrund auf.

Hinweis: `mykeyvault-mcp` läuft produktiv als HTTP-MCP-Container (`:3458`, `https://keyvault-mcp.lan`). Der Repo-Code (`mcp/src/index.ts`) zeigt noch die ältere stdio-Variante — das Repo hängt hier hinter der Produktion. Dieses Diagramm bildet die deployte Realität ab.